

# Wisconsin Robotics

# URC LiDAR Path Planning System

Technical Documentation

---

University of Wisconsin-Madison  
Wisconsin Robotics

Shreyas Rao Karbar

## 0 Contents

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Project Overview</b>                                               | <b>2</b>  |
| 1.1      | Purpose . . . . .                                                     | 2         |
| 1.2      | Problem Statement . . . . .                                           | 2         |
| 1.3      | Rover Physical Constraints . . . . .                                  | 2         |
| <b>2</b> | <b>System Architecture</b>                                            | <b>2</b>  |
| 2.1      | Pipeline Overview . . . . .                                           | 2         |
| 2.2      | Module Map . . . . .                                                  | 3         |
| <b>3</b> | <b>Configuration</b>                                                  | <b>3</b>  |
| <b>4</b> | <b>Module Documentation</b>                                           | <b>4</b>  |
| 4.1      | LiDAR Loading and Cleaning: <code>load_clean.py</code> . . . . .      | 4         |
| 4.2      | KNN Builder: <code>knn_builder.py</code> . . . . .                    | 7         |
| 4.3      | Graph Builder: <code>graph_builder.py</code> . . . . .                | 7         |
| 4.4      | A* Path Planning: <code>astar.py</code> . . . . .                     | 8         |
| 4.5      | Multi-Target Planning: <code>multi_target.py</code> . . . . .         | 9         |
| 4.6      | GPS Coordinate Conversion: <code>point_conversion.py</code> . . . . . | 9         |
| 4.7      | Nearest Node Snapping: <code>nearest_point.py</code> . . . . .        | 10        |
| 4.8      | Visualization: <code>visualization.py</code> . . . . .                | 10        |
| <b>5</b> | <b>Testing Utilities</b>                                              | <b>10</b> |
| 5.1      | <code>testing/test_pipeline.py</code> . . . . .                       | 10        |
| 5.2      | <code>testing/graph_stats.py</code> . . . . .                         | 11        |
| <b>6</b> | <b>Running the System</b>                                             | <b>11</b> |
| 6.1      | Prerequisites . . . . .                                               | 11        |
| 6.2      | Full Pipeline . . . . .                                               | 11        |
| 6.3      | Benchmark Only . . . . .                                              | 11        |
| <b>7</b> | <b>Algorithm Complexity Summary</b>                                   | <b>12</b> |
| <b>8</b> | <b>Coordinate System Notes</b>                                        | <b>12</b> |
| <b>9</b> | <b>Test Run Results</b>                                               | <b>12</b> |
| 9.1      | Waypoints Used . . . . .                                              | 12        |
| 9.2      | Path Visualization . . . . .                                          | 13        |
| 9.3      | Path Statistics . . . . .                                             | 13        |

## 1 Project Overview

---

### 1.1 Purpose

This system computes an optimal, slope-aware path across real-world Mars Desert Research Station (MDRS) terrain for the Wisconsin Robotics rover competing in the University Rover Challenge (URC). Given a LiDAR scan of the terrain and a set of GPS waypoints, it returns the safest ordered route visiting all targets.

### 1.2 Problem Statement

The URC Autonomous Navigation task requires the rover to autonomously visit multiple GPS waypoints in the field. The terrain at MDRS (Hanksville, Utah) contains arroyos, rock formations, and slopes that can tip or damage the rover. The planner must:

- Process raw LiDAR point cloud data into a navigable terrain graph
- Convert GPS waypoints (lat/lon) into the same coordinate system as the LiDAR data
- Find the globally optimal visit order across all targets (Travelling Salesman Problem)
- Return a slope-safe path that the rover can physically traverse

### 1.3 Rover Physical Constraints

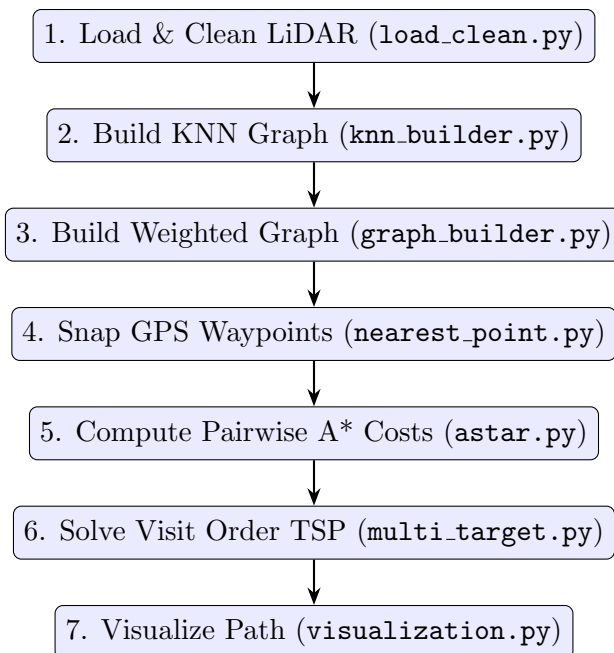
| Parameter                      | Value                       |
|--------------------------------|-----------------------------|
| Wheel width                    | 8 in ( $\approx 0.203$ m)   |
| Wheel diameter                 | 8.5 in ( $\approx 0.216$ m) |
| Ground clearance (with arm)    | 8.5 in ( $\approx 0.216$ m) |
| Ground clearance (without arm) | 13 in ( $\approx 0.330$ m)  |
| Rover width                    | 31 in ( $\approx 0.787$ m)  |
| Rover length                   | 39 in ( $\approx 0.991$ m)  |
| Maximum traversable slope      | 36                          |

## 2 System Architecture

---

### 2.1 Pipeline Overview

The system runs as a linear pipeline. Each stage produces output consumed directly by the next.



## 2.2 Module Map

| File                | Package          | Responsibility                               |
|---------------------|------------------|----------------------------------------------|
| main.py             | root             | Entry point, orchestration                   |
| load_clean.py       | src/pointcloud   | LiDAR I/O, voxel filter, outlier removal     |
| knn_builder.py      | src/pointcloud   | KDTree construction, KNN queries             |
| graph_builder.py    | src/pathplanning | Slope-weighted adjacency list                |
| astar.py            | src/pathplanning | A* shortest path search                      |
| multi_target.py     | src/pathplanning | Cost matrix, TSP brute-force, path stitching |
| geo_helpers.py      | src/utils        | Vectorized edge weight computation           |
| nearest_point.py    | src/utils        | 2D GPS-to-graph node snapping                |
| point_conversion.py | src/utils        | EPSG extraction, GPS coordinate transform    |
| visualization.py    | src/utils        | Folium satellite map output                  |
| settings.py         | configs          | Global tunable constants                     |
| test_pipeline.py    | testing          | Timing benchmark (no path, no GPS)           |
| graph_stats.py      | testing          | Graph connectivity statistics                |

## 3 Configuration

All global constants live in `configs/settings.py`. Change them here and every module picks up the new values automatically.



| Constant         | Default | Effect                                                             |
|------------------|---------|--------------------------------------------------------------------|
| K_NEIGHBORS      | 10      | Number of KNN edges per node. Higher = denser graph, slower build. |
| MAX_SLOPE_DEG    | 36      | Edges steeper than this are removed entirely.                      |
| SLOPE_MULTIPLIER | 5       | Scales slope penalty in edge weight (see §4.3).                    |

## 4 Module Documentation

### 4.1 LiDAR Loading and Cleaning: `load_clean.py`

**Function:** `load_and_clean_lidar(path, voxel_size=0.35, min_points_per_cell=2)`

**Input:** Path to a `.laz` or `.las` file.

**Output:** NumPy array of shape  $(M, 3)$ , the cleaned terrain point cloud.

#### Voxel Hash Filter

The raw LiDAR scan can contain tens of millions of points. Running A\* over a graph of that size is computationally infeasible. The voxel hash filter downsamples to one representative point per  $0.35 \text{ m} \times 0.35 \text{ m}$  ground cell.

#### Voxel Size Selection

**Objective.** The goal of this analysis is to determine an appropriate voxel size for downsampling LiDAR terrain data used in rover path planning. The voxel size must preserve terrain features that affect rover drivability while reducing noise and computational complexity for graph construction and path planning.

#### Rover parameters used in the analysis.

- Wheel diameter: 8.5 in ( $\approx 0.216 \text{ m}$ ), wheel radius: 0.108 m
- Wheel width: 8 in ( $\approx 0.203 \text{ m}$ )
- Ground clearance (with arm): 8.5 in ( $\approx 0.216 \text{ m}$ )
- Maximum climbable slope: 36

Rover length and width are not directly used for voxel size determination, as they are more relevant for later footprint-based collision checking.

**Core principle.** A rover cannot meaningfully respond to terrain features smaller than the physical scale of its wheel-ground interaction. Therefore, the voxel size should reflect the minimum terrain feature size that can influence rover mobility, such as slope, grounding risk, or loss of traction.

**Vertical terrain constraint.** The rover risks grounding when vertical terrain variation exceeds its ground clearance. To maintain a safety margin, vertical variations are limited to a fraction of the ground clearance:

$$\Delta z_{\text{safe}} \leq (0.3\text{--}0.5) \times \text{ground clearance} \quad (1)$$

$$\Delta z_{\text{safe}} = 0.4 \times 0.216 \text{ m} \approx 0.086 \text{ m} \quad (2)$$

Vertical changes smaller than approximately 8.6 cm can therefore be safely smoothed.

**Slope-based horizontal resolution.** Using the maximum allowable slope  $\tan(\theta) = \Delta z / \Delta x$ , solving for the minimum horizontal cell width:

$$\Delta x = \frac{\Delta z_{\text{safe}}}{\tan(36)} \quad (3)$$

Since  $\tan(36) \approx 0.726$ :

$$\Delta x = \frac{0.086}{0.726} \approx 0.118 \text{ m} \quad (4)$$

This implies that horizontal terrain features smaller than approximately 12 cm cannot generate slopes critical to rover operation.

**Wheel width constraint.** The voxel size must not be smaller than the wheel width. If the voxel size is less than the wheel width, the terrain model may capture features affecting only a single wheel, which is incompatible with point-based graph representations.

$$\text{voxel size} \geq \text{wheel width} \approx 0.20 \text{ m} \quad (5)$$

**Lower bound summary.**

| Constraint                   | Minimum Value |
|------------------------------|---------------|
| Slope realism                | 0.12 m        |
| Vertical clearance smoothing | 0.15 m        |
| Wheel width                  | 0.20 m        |

Following robotics best practices, the voxel size is chosen to be 1.5–2× the strict minimum to provide robustness against noise and modeling error.

**Recommended voxel size.**

$$\boxed{\text{Voxel size} \approx 0.35 \text{ m}} \quad (6)$$

This value:

- Exceeds wheel width, avoiding false sub-wheel-scale obstacles
- Preserves slopes near the rover’s maximum capability (36)
- Smooths sub-wheel-scale sensor noise
- Significantly reduces graph size and computation time

**Conclusion.** The selected voxel size of 0.35 m is grounded in rover kinematics, terrain interaction physics, and slope constraints. This choice ensures that the downsampled terrain remains physically meaningful for path planning while enabling efficient graph construction and fast A\* execution.

### Voxel Hash Algorithm

Each LiDAR point  $(x, y, z)$  is mapped to integer grid coordinates:

$$c_x = \left\lfloor \frac{x}{v} \right\rfloor, \quad c_y = \left\lfloor \frac{y}{v} \right\rfloor$$

where  $v = 0.35$  is the voxel size. The pair  $(c_x, c_y)$  is encoded as a single collision-free integer key:

$$k = (c_x - c_{x,\min}) \cdot (c_{y,\max} - c_{y,\min} + 1) + (c_y - c_{y,\min})$$

The stride-based encoding avoids hash collisions regardless of absolute UTM coordinates (which can reach  $\sim 4\,200\,000$  m northing). Points are sorted by key in a single `numpy` argsort, making all points in the same cell contiguous, with no hash map required.

### Outlier Removal

Cells with fewer than `min_points_per_cell` (default 2) points are discarded. Isolated single-return cells are almost always sensor noise or scan-edge artifacts. This achieves the same effect as statistical outlier removal but runs in  $O(N)$  instead of  $O(N \log N)$ .

### Representative Point Selection

For each valid cell, the point with the *highest*  $z$  value is kept. This is a real measured LiDAR return, not a centroid. Using max- $z$  ensures slopes are never underestimated: averaging  $z$  would make steep cells appear flatter than they physically are.

## Complexity

| Operation          | Complexity    |
|--------------------|---------------|
| Grid assignment    | $O(N)$        |
| Key encoding       | $O(N)$        |
| Sorting by key     | $O(N \log N)$ |
| Cell boundary scan | $O(N)$        |
| Max- $z$ selection | $O(N)$        |

### 4.2 KNN Builder: knn\_builder.py

**Function:** `build_knn(points)`

**Input:** Cleaned point cloud  $(M, 3)$ .

**Output:** `neighbour_indices`  $(M, K)$ , `neighbour_distances`  $(M, K)$ , `tree` (KDTree).

A `scipy.spatial.cKDTree` is built over the 3D point cloud. For each point,  $K + 1$  nearest neighbours are queried (the extra 1 accounts for the point being its own nearest neighbour). The self-neighbour is stripped, yielding arrays of shape  $(M, K)$ . The query uses `workers=-1` to parallelize across all CPU cores.

**Why a 3D KDTree for a terrain graph?** Building the KDTree in 3D prevents points on opposite faces of a steep cliff from being linked as neighbours. A 2D KDTree would connect them (they are close horizontally) even though the rover cannot traverse the vertical drop between them.

### 4.3 Graph Builder: graph\_builder.py

**Function:** `build_graph_vectorized(points, neighbour_indices, neighbour_distances)`

**Input:** Point cloud, KNN indices, KNN distances.

**Output:** Adjacency list where `graph[i]` is a list of `(neighbour_index, weight)` tuples.

## Edge Weight Formula

For each candidate edge  $(i, j)$  with Euclidean distance  $d$ , the terrain slope is:

$$\theta = \arctan \left( \frac{|\Delta z|}{\max(\sqrt{\Delta x^2 + \Delta y^2}, 10^{-6})} \right)$$

The edge weight is then:

$$w = \begin{cases} d \cdot \left( 1 + \frac{\theta}{\theta_{\max}} \cdot S \right) & \text{if } \theta \leq \theta_{\max} \\ \text{rejected (NaN)} & \text{if } \theta > \theta_{\max} \end{cases}$$

where  $\theta_{\max} = 36$  (`MAX_SLOPE_DEG`) and  $S = 5$  (`SLOPE_MULTIPLIER`).

A perfectly flat edge costs exactly  $d$ . An edge at the maximum slope costs  $d \cdot (1 + S) = 6d$ . Edges exceeding  $\theta_{\max}$  are removed entirely, making those terrain passages impassable in the graph.

## Vectorized Implementation

All  $M \times K$  edges are processed simultaneously as NumPy arrays. Duplicate edges arising from symmetric KNN queries are deduplicated with `np.unique`. The adjacency list is assembled by sorting all directed edges by source-node index and slicing per-node with a precomputed `indptr` array, with no Python-level `append` loop.

### 4.4 A\* Path Planning: `astar.py`

**Function:** `astar(graph, points, start_idx, goal_idx)`

**Input:** Adjacency list, point cloud, start and goal node indices.

**Output:** (`path`, `cost`), a node index list and total cost. Returns (`None`, `inf`) if unreachable.

## Heuristic

The heuristic is the straight-line 3D Euclidean distance from node  $n$  to the goal:

$$h(n) = \|\mathbf{p}_n - \mathbf{p}_{\text{goal}}\|_2$$

This heuristic is *admissible*: it never overestimates the true cost because every edge weight  $w \geq d \geq h$  (the slope penalty only increases  $w$  above the Euclidean distance  $d$ ). An admissible heuristic guarantees that A\* returns the globally optimal path.

## Data Structures

- **Open set:** heapq min-heap of (`f_score`, `node_index`) tuples.
- **g-scores:** dictionary; unvisited nodes are implicitly  $\infty$ .
- **came\_from:** dictionary for backtracking path reconstruction.
- **Visited set:** nodes popped from the heap are never re-expanded.

## Path Reconstruction

When the goal is popped from the heap, the path is reconstructed by following `came_from` backwards from goal to start, then reversing the result.

#### 4.5 Multi-Target Planning: `multi_target.py`

This module solves the problem of visiting  $N$  target waypoints in the lowest-cost order starting from a fixed start node.

**Cost Matrix:** `compute_cost_matrix`

**Input:** Graph, point cloud, list of  $n$  node indices (start + all targets).

**Output:**  $n \times n$  cost matrix and path matrix.

$A^*$  is run between every pair of the  $n$  nodes. Only the upper triangle ( $i < j$ ) is computed since the graph is undirected and costs are symmetric:

$$\text{cost\_matrix}[i][j] = \text{cost\_matrix}[j][i] = A^*(\text{node}_i, \text{node}_j)$$

This produces  $\binom{n}{2}$   $A^*$  calls total.

**Optimal Visit Order:** `find_best_order`

**Input:** Cost matrix.

**Output:** Best permutation of target indices, total cost.

All  $N!$  orderings of the  $N$  targets are enumerated with `itertools.permutations`. The start node (index 0) is always fixed; only the target visit order varies. The lowest-cost feasible permutation is returned. This is an exact TSP solution, optimal for all  $N \leq 10$ . A guard raises `ValueError` for  $N > 10$  to prevent excessive runtime (URC uses  $N \leq 5$  in practice).

**Path Stitching:** `build_full_path`

**Input:** Path matrix, best visit order.

**Output:** Single flat list of node indices for the complete route.

Segments are concatenated in order. The first node of each segment (equal to the last node of the previous segment) is dropped at join points to avoid duplicates.

#### 4.6 GPS Coordinate Conversion: `point_conversion.py`

`get_epsg(laz_path)`

Opens the LAZ file in streaming mode (`laspy.open`) to read only the header. No point data is loaded. Returns the integer EPSG code of the file's coordinate reference system (e.g., 32612 for UTM Zone 12N, which covers the MDRS site).

`gps_to_xy(lat, lon, epsg)`

Converts a WGS84 GPS coordinate (lat/lon) into the same projected horizontal coordinates as the LiDAR data using `pyproj.Transformer`. Returns only  $(x, y)$ ; altitude is discarded because:

1. Competition waypoints are provided as lat/lon only, with no altitude.
2. GPS ellipsoidal altitude (WGS84) is incompatible with LiDAR  $z$ , which uses a local vertical datum. The correct  $z$  is always obtained from the terrain by snapping to the nearest graph node.

Transformers are cached in `_transformer_cache` (keyed by EPSG code) to avoid rebuilding per waypoint.

#### 4.7 Nearest Node Snapping: `nearest_point.py`

**Function:** `find_nearest_node(xy, points_array, graph)`

**Input:** 2D projected coordinate `[x, y]`, full point cloud, adjacency list.

**Output:** (`node_index`, `3D_point`) of the nearest connected node.

Snapping is done in 2D (horizontal only, ignoring  $z$ ) because the GPS provides only horizontal position and including  $z$  would cause incorrect snapping to nodes on cliffs directly below the query point.

The function iterates through the  $K$  nearest 2D candidates (closest first) and returns the first with at least one graph edge. A node with no edges would immediately trap the rover; the first *connected* candidate is always preferred. If every candidate is isolated (extremely unlikely), the geometrically closest is returned as a fallback.

#### 4.8 Visualization: `visualization.py`

**Function:** `visualize_path(full_path, points, start_idx, target_nodes, epsg)`

Generates an interactive HTML file (`path_output.html`) using `folium` with an Esri World Imagery satellite tile layer. Contents:

- Red polyline tracing the complete rover path
- Green circle marker at the start position
- Orange circle markers at each target waypoint

Each node's projected  $(x, y)$  is back-transformed to WGS84 lat/lon for folium display. The reverse transformer is cached using the same `_transformer_cache` pattern as `point_conversion.py`.

## 5 Testing Utilities

### 5.1 `testing/test_pipeline.py`

Benchmarks the three most expensive stages (`load_and_clean_lidar`, `build_knn`, and `build_graph_vector`) printing wall-clock time for each, then graph statistics. Does not require GPS input.

```
python testing/test_pipeline.py
```

## 5.2 testing/graph\_stats.py

Computes and prints:

- Total nodes and edges
- Min, max, and mean node degree
- Number of isolated nodes (degree 0)

A high isolated-node count indicates terrain areas where every connection was rejected due to excessive slope, and may suggest `MAX_SLOPE_DEG` is set too low for the terrain being navigated.

## 6 Running the System

### 6.1 Prerequisites

```
pip install laspy numpy scipy pyproj folium
```

### 6.2 Full Pipeline

From the repository root:

```
python main.py
```

Prompts (in order):

1. Path to the `.laz/.las` LiDAR file (e.g. `data/mdrs.laz`)
2. Start GPS: `lat lon` in decimal degrees, space-separated
3. Number of target waypoints  $N$
4. GPS for each target: `lat lon`

Output: path saved to `path_output.html`, slope statistics printed to terminal.

### 6.3 Benchmark Only

```
python testing/test_pipeline.py
```

Prompts only for the LiDAR file path. Prints per-stage timing and graph statistics.



## 7 Algorithm Complexity Summary

| Stage           | Time Complexity                             | Notes                    |
|-----------------|---------------------------------------------|--------------------------|
| Voxel filter    | $O(N \log N)$                               | Dominated by argsort     |
| KDTree build    | $O(M \log M)$                               | scipy cKDTree            |
| KNN query       | $O(MK \log M)$                              | Parallelized             |
| Graph build     | $O(MK)$                                     | Fully vectorized         |
| Single A* call  | $O(E \log V)$                               | $E$ = edges, $V$ = nodes |
| Cost matrix     | $O\left(\binom{n}{2} \cdot E \log V\right)$ | $n$ = waypoints          |
| TSP brute force | $O(n!)$                                     | $n \leq 10$              |

$N$ : raw LiDAR point count (millions).  $M$ : downsampled node count (tens of thousands).  
 $n$ : number of waypoints ( $\leq 5$  for URC).

## 8 Coordinate System Notes

- **LiDAR data** is stored in a projected CRS (UTM). All  $x, y$  values are in metres;  $z$  is orthometric height in metres.
- **GPS input** is WGS84 decimal degrees (EPSG:4326), converted to the LAZ CRS via `pyproj`.
- **Folium output** requires WGS84. Path nodes are back-projected from the LAZ CRS to lat/lon before rendering.
- `always_xy=True` is set on every `pyproj.Transformer` to enforce consistent (longitude, latitude) / (x, y) axis ordering and prevent the classic lat/lon swap bug.

## 9 Test Run Results

The following results were produced running the full pipeline on `data/mdrs.laz` with one start point and three target waypoints at the Mars Desert Research Station (MDRS), Hanksville, Utah.

### 9.1 Waypoints Used

| Point    | Latitude  | Longitude   |
|----------|-----------|-------------|
| Start    | 38.406500 | -110.791900 |
| Target 1 | 38.409195 | -110.788461 |
| Target 2 | 38.403000 | -110.785000 |
| Target 3 | 38.407500 | -110.782000 |

## 9.2 Path Visualization

The optimal visit order computed by the TSP solver was Start  $\rightarrow$  Target 1  $\rightarrow$  Target 3  $\rightarrow$  Target 2, with a total path cost of 2535.59. The red polyline shows the full 923-node route overlaid on Esri satellite imagery. The green marker is the start; orange markers are the three targets.

## 9.3 Path Statistics

| Metric            | Value   |
|-------------------|---------|
| Best path cost    | 2535.59 |
| Total path nodes  | 923     |
| Maximum slope     | 15.1°   |
| Mean slope        | 1.4°    |
| Segments over 20° | 0       |

The maximum slope of 15.1° is well within the rover's 36° traversability limit. Zero segments exceed 20°, confirming the planner actively avoids steep terrain even when shorter steep routes exist.



Figure 1: Planned rover path on MDRS terrain. Red line: A\* route. Green: start. Orange: targets.

```
Best path cost : 2535.59
Full path has 923 nodes
Path slope — max: 15.1° mean: 1.4° segments over 20°: 0
```

Figure 2: Terminal output showing path cost, node count, and slope analysis.